

Chapter 10

Syntax Reference

10.1 Notational Conventions

These notational conventions are used for presenting syntax:

<i>[pattern]</i>	optional
<i>{pattern}</i>	zero or more repetitions
<i>(pattern)</i>	grouping
<i>pat₁ pat₂</i>	choice
<i>pat_{<pat'>}</i>	difference—elements generated by <i>pat</i> except those generated by <i>pat'</i>
<i>fibonacci</i>	terminal syntax in typewriter font

BNF-like syntax is used throughout, with productions having the form:

$$\text{nonterm} \rightarrow \text{alt}_1 \mid \text{alt}_2 \mid \dots \mid \text{alt}_n$$

In both the lexical and the context-free syntax, there are some ambiguities that are to be resolved by making grammatical phrases as long as possible, proceeding from left to right (in shift-reduce parsing, resolving shift/reduce conflicts by shifting). In the lexical syntax, this is the “maximal munch” rule. In the context-free syntax, this means that conditionals, let-expressions, and lambda abstractions extend to the right as far as possible.

10.2 Lexical Syntax

<i>program</i>	→ { <i>lexeme</i> <i>whitespace</i> }
<i>lexeme</i>	→ <i>qvarid</i> <i>qconid</i> <i>qvarsym</i> <i>qconsym</i> <i>literal</i> <i>special</i> <i>reservedop</i> <i>reservedid</i>
<i>literal</i>	→ <i>integer</i> <i>float</i> <i>char</i> <i>string</i>
<i>special</i>	→ () , ; [] ` { }
<i>whitespace</i>	→ <i>whitestuff</i> { <i>whitestuff</i> }
<i>whitestuff</i>	→ <i>whitechar</i> <i>comment</i> <i>ncomment</i>
<i>whitechar</i>	→ <i>newline</i> <i>vertab</i> <i>space</i> <i>tab</i> <i>uniWhite</i>
<i>newline</i>	→ <i>return</i> <i>linefeed</i> <i>return</i> <i>linefeed</i> <i>formfeed</i>
<i>return</i>	→ a carriage return
<i>linefeed</i>	→ a line feed
<i>vertab</i>	→ a vertical tab
<i>formfeed</i>	→ a form feed
<i>space</i>	→ a space
<i>tab</i>	→ a horizontal tab
<i>uniWhite</i>	→ any Unicode character defined as whitespace
<i>comment</i>	→ <i>dashes</i> [<i>any</i> _{<symbol>} { <i>any</i> }] <i>newline</i>
<i>dashes</i>	→ -- { - }
<i>opencom</i>	→ { -
<i>closecom</i>	→ - }
<i>ncomment</i>	→ <i>opencom</i> <i>ANY seq</i> { <i>ncomment</i> <i>ANY seq</i> } <i>closecom</i>
<i>ANY seq</i>	→ { <i>ANY</i> } { { <i>ANY</i> } (<i>opencom</i> <i>closecom</i>) { <i>ANY</i> } }
<i>ANY</i>	→ <i>graphic</i> <i>whitechar</i>
<i>any</i>	→ <i>graphic</i> <i>space</i> <i>tab</i>
<i>graphic</i>	→ <i>small</i> <i>large</i> <i>symbol</i> <i>digit</i> <i>special</i> " '

<i>small</i>	→	<i>ascSmall</i> <i>uniSmall</i> _	
<i>ascSmall</i>	→	a b ... z	
<i>uniSmall</i>	→	any Unicode lowercase letter	
<i>large</i>	→	<i>ascLarge</i> <i>uniLarge</i>	
<i>ascLarge</i>	→	A B ... Z	
<i>uniLarge</i>	→	any uppercase or titlecase Unicode letter	
<i>symbol</i>	→	<i>ascSymbol</i> <i>uniSymbol</i> _{<i>special</i> _ " ' }	
<i>ascSymbol</i>	→	! # \$ % & * + . / < = > ? @ \ ^ - ~ :	
<i>uniSymbol</i>	→	any Unicode symbol or punctuation	
<i>digit</i>	→	<i>ascDigit</i> <i>uniDigit</i>	
<i>ascDigit</i>	→	0 1 ... 9	
<i>uniDigit</i>	→	any Unicode decimal digit	
<i>octit</i>	→	0 1 ... 7	
<i>hexit</i>	→	<i>digit</i> A ... F a ... f	
<i>varid</i>	→	(<i>small</i> { <i>small</i> <i>large</i> <i>digit</i> ' }) _{<i>reservedid</i>}	
<i>conid</i>	→	<i>large</i> { <i>small</i> <i>large</i> <i>digit</i> ' }	
<i>reservedid</i>	→	case class data default deriving do else foreign if import in infix infixl infixr instance let module newtype of then type where _	
<i>varsym</i>	→	(<i>symbol</i> _{,} { <i>symbol</i> }) _{<i>reservedop</i> dashes}	
<i>consym</i>	→	(: { <i>symbol</i> }) _{<i>reservedop</i>}	
<i>reservedop</i>	→	.. : :: = \ <- -> @ ~ =>	
<i>varid</i>		(variables)	
<i>conid</i>		(constructors)	
<i>tyvar</i>	→	<i>varid</i>	(type variables)
<i>tycon</i>	→	<i>conid</i>	(type constructors)
<i>tycls</i>	→	<i>conid</i>	(type classes)
<i>modid</i>	→	{ <i>conid</i> . } <i>conid</i>	(modules)
<i>qvarid</i>	→	[<i>modid</i> .] <i>varid</i>	
<i>qconid</i>	→	[<i>modid</i> .] <i>conid</i>	
<i>qtycon</i>	→	[<i>modid</i> .] <i>tycon</i>	
<i>qtycls</i>	→	[<i>modid</i> .] <i>tycls</i>	
<i>qvarsym</i>	→	[<i>modid</i> .] <i>varsym</i>	
<i>qconsym</i>	→	[<i>modid</i> .] <i>consym</i>	
<i>decimal</i>	→	<i>digit</i> { <i>digit</i> }	
<i>octal</i>	→	<i>octit</i> { <i>octit</i> }	
<i>hexadecimal</i>	→	<i>hexit</i> { <i>hexit</i> }	
<i>integer</i>	→	<i>decimal</i> 0o <i>octal</i> 0O <i>octal</i> 0x <i>hexadecimal</i> 0X <i>hexadecimal</i>	
<i>float</i>	→	<i>decimal</i> . <i>decimal</i> [<i>exponent</i>] <i>decimal</i> <i>exponent</i>	
<i>exponent</i>	→	(e E) [+ -] <i>decimal</i>	
<i>char</i>	→	' (<i>graphic</i> _{,} \ <i>space</i> <i>escape</i> _{\&}) '	
<i>string</i>	→	" { <i>graphic</i> _{,} \ <i>space</i> <i>escape</i> <i>gap</i> } "	

<i>escape</i>	→ \ (<i>charesc</i> <i>ascii</i> <i>decimal</i> <i>o</i> <i>octal</i> <i>x</i> <i>hexadecimal</i>)
<i>charesc</i>	→ a b f n r t v \ " ' &
<i>ascii</i>	→ ^ <i>cntrl</i> NUL SOH STX ETX EOT ENQ ACK BEL BS HT LF VT FF CR SO SI DLE DC1 DC2 DC3 DC4 NAK SYN ETB CAN EM SUB ESC FS GS RS US SP DEL
<i>cntrl</i>	→ <i>ascLarge</i> @ [\] ^ _
<i>gap</i>	→ \ <i>whitechar</i> { <i>whitechar</i> } \

10.3 Layout

Section 2.7 gives an informal discussion of the layout rule. This section defines it more precisely.

The meaning of a Haskell program may depend on its *layout*. The effect of layout on its meaning can be completely described by adding braces and semicolons in places determined by the layout. The meaning of this augmented program is now layout insensitive.

The effect of layout is specified in this section by describing how to add braces and semicolons to a laid-out program. The specification takes the form of a function L that performs the translation. The input to L is:

- A stream of lexemes as specified by the lexical syntax in the Haskell report, with the following additional tokens:
 - If a `let`, `where`, `do`, or `of` keyword is not followed by the lexeme `{`, the token $\{n\}$ is inserted after the keyword, where n is the indentation of the next lexeme if there is one, or 0 if the end of file has been reached.
 - If the first lexeme of a module is not `{` or `module`, then it is preceded by $\{n\}$ where n is the indentation of the lexeme.
 - Where the start of a lexeme is preceded only by white space on the same line, this lexeme is preceded by $<n>$ where n is the indentation of the lexeme, provided that it is not, as a consequence of the first two rules, preceded by $\{n\}$. (NB: a string literal may span multiple lines – Section 2.6. So in the fragment

```
f = ("Hello \
    \Bill", "Jake")
```

There is no $<n>$ inserted before the `\Bill`, because it is not the beginning of a complete lexeme; nor before the `,`, because it is not preceded only by white space.)

- A stack of “layout contexts”, in which each element is either:
 - Zero, indicating that the enclosing context is explicit (i.e. the programmer supplied the opening brace). If the innermost context is 0, then no layout tokens will be inserted until either the enclosing context ends or a new context is pushed.
 - A positive integer, which is the indentation column of the enclosing layout context.

The “indentation” of a lexeme is the column number of the first character of that lexeme; the indentation of a line is the indentation of its leftmost lexeme. To determine the column number, assume a fixed-width font with the following conventions:

- The characters *newline*, *return*, *linefeed*, and *formfeed*, all start a new line.
- The first column is designated column 1, not 0.
- Tab stops are 8 characters apart.
- A tab character causes the insertion of enough spaces to align the current position with the next tab stop.

For the purposes of the layout rule, Unicode characters in a source program are considered to be of the same, fixed, width as an ASCII character. However, to avoid visual confusion, programmers should avoid writing programs in which the meaning of implicit layout depends on the width of non-space characters.

The application $L\ tokens\ []$ delivers a layout-insensitive translation of *tokens*, where *tokens* is the result of lexically analysing a module and adding column-number indicators to it as described above. The definition of L is as follows, where we use “.” as a stream construction operator, and “[]” for the empty stream.

$L(< n >: ts) (m : ms)$	$= ; : (L ts (m : ms))$	if $m = n$
	$= } : (L (< n >: ts) ms)$	if $n < m$
$L(< n >: ts) ms$	$= L ts ms$	
$L(\{n\} : ts) (m : ms)$	$= \{ : (L ts (n : m : ms))$	if $n > m$ (Note 1)
$L(\{n\} : ts) []$	$= \{ : (L ts [n])$	if $n > 0$ (Note 1)
$L(\{n\} : ts) ms$	$= \{ : } : (L (< n >: ts) ms)$	(Note 2)
$L\{ : ts\} (0 : ms)$	$= } : (L ts ms)$	(Note 3)
$L\{ : ts\} ms$	$= \text{parse-error}$	(Note 3)
$L\{ : ts\} ms$	$= \{ : (L ts (0 : ms))$	(Note 4)
$L(t : ts) (m : ms)$	$= } : (L (t : ts) ms)$	if $m' = 0$ and $\text{parse-error}(t)$ (Note 5)
$L(t : ts) ms$	$= t : (L ts ms)$	
$L [] []$	$= []$	
$L [] (m : ms)$	$= } : L [] ms$	if $m \neq 0$ (Note 6)

Note 1.

A nested context must be further indented than the enclosing context ($n > m$). If not, L fails, and the compiler should indicate a layout error. An example is:

```
f x = let
      h y = let
        p z = z
      in h
in p
```

Here, the definition of p is indented less than the indentation of the enclosing context, which is set in this case by the definition of h .

Note 2.

If the first token after a where (say) is not indented more than the enclosing layout context, then the block must be empty, so empty braces are inserted. The $\{n\}$ token is replaced by $< n >$, to mimic the situation if the empty braces had been explicit.

Note 3.

By matching against 0 for the current layout context, we ensure that an explicit close brace can only match an explicit open brace. A parse error results if an explicit close brace matches an implicit open brace.

Note 4.

This clause means that all brace pairs are treated as explicit layout contexts, including labelled construction and update (Section 3.15). This is a difference between this formulation and Haskell 1.4.

Note 5.

The side condition $\text{parse-error}(t)$ is to be interpreted as follows: if the tokens generated so far by L together with the next token t represent an invalid prefix of the Haskell grammar, and the tokens generated so far by L followed by the token “ $\}$ ” represent a valid prefix of the Haskell grammar, then $\text{parse-error}(t)$ is true.

The test $m' = 0$ checks that an implicitly-added closing brace would match an implicit open brace.

Note 6.

At the end of the input, any pending close-braces are inserted. It is an error at this point to be within a non-layout context (i.e. $m = 0$).

If none of the rules given above matches, then the algorithm fails. It can fail for instance when the end of the input is reached, and a non-layout context is active, since the close brace is missing. Some error conditions are not detected by the algorithm, although they could be: for example `let x = e; y = x in e'`.

Note 1 implements the feature that layout processing can be stopped prematurely by a parse error. For example

```
let x = e; y = x in e'
```

is valid, because it translates to

```
let { x = e; y = x } in e'
```

The close brace is inserted due to the parse error rule above.

10.4 Literate comments

The “literate comment” convention, first developed by Richard Bird and Philip Wadler for Orwell, and inspired in turn by Donald Knuth’s “literate programming”, is an alternative style for encoding Haskell source code. The literate style encourages comments by making them the default. A line in which “>” is the first character is treated as part of the program; all other lines are comments.

The program text is recovered by taking only those lines beginning with “>”, and replacing the leading “>” with a space. Layout and comments apply exactly as described in Chapter [10](#) in the resulting text.

To capture some cases where one omits an “>” by mistake, it is an error for a program line to appear adjacent to a non-blank comment line, where a line is taken as blank if it consists only of whitespace.

By convention, the style of comment is indicated by the file extension, with “.hs” indicating a usual Haskell file and “.lhs” indicating a literate Haskell file. Using this style, a simple factorial program would be:

This literate program prompts the user for a number
and prints the factorial of that number:

```
> main :: IO ()

> main = do putStr "Enter a number: "
>           l <- readLine
>           putStr "n!= "
>           print (fact (read l))
```

This is the factorial function.

```
> fact :: Integer -> Integer
> fact 0 = 1
> fact n = n * fact (n-1)
```

An alternative style of literate programming is particularly suitable for use with the LaTeX text processing system. In this convention, only those parts of the literate program that are entirely enclosed between `\begin{code}...` delimiters are treated as program text; all other lines are comments. More precisely:

- Program code begins on the first line following a line that begins `\begin{code}`.
- Program code ends just before a subsequent line that begins `\end{code}` (ignoring string literals, of course).

It is not necessary to insert additional blank lines before or after these delimiters, though it may be stylistically desirable. For example,

```
\documentstyle{article}

\begin{document}

\chapter{Introduction}

This is a trivial program that prints the first 20 factorials.

\begin{code}
main :: IO ()
main = print [ (n, product [1..n]) | n <- [1..20]]
\end{code}

\end{document}
```

This style uses the same file extension. It is not advisable to mix these two styles in the same file.

10.5 Context-Free Syntax

<i>module</i>	→ <i>module modid</i> [<i>exports</i>] <i>where body</i>	
	<i>body</i>	
<i>body</i>	→ { <i>impdecls</i> ; <i>topdecls</i> }	
	{ <i>impdecls</i> }	
	{ <i>topdecls</i> }	
<i>impdecls</i>	→ <i>impdecl</i> ₁ ; ... ; <i>impdecl</i> _{<i>n</i>}	(<i>n</i> ≥ 1)
<i>exports</i>	→ (<i>export</i> ₁ , ... , <i>export</i> _{<i>n</i>} [,])	(<i>n</i> ≥ 0)
<i>export</i>	→ <i>qvar</i>	
	<i>qtycon</i> [(..) (<i>cname</i> ₁ , ... , <i>cname</i> _{<i>n</i>})]	(<i>n</i> ≥ 0)
	<i>qtycls</i> [(..) (<i>qvar</i> ₁ , ... , <i>qvar</i> _{<i>n</i>})]	(<i>n</i> ≥ 0)
	<i>module modid</i>	
<i>impdecl</i>	→ <i>import</i> [<i>qualified</i>] <i>modid</i> [<i>as modid</i>] [<i>impspec</i>]	
		(empty declaration)
<i>impspec</i>	→ (<i>import</i> ₁ , ... , <i>import</i> _{<i>n</i>} [,])	(<i>n</i> ≥ 0)
	<i>hiding</i> (<i>import</i> ₁ , ... , <i>import</i> _{<i>n</i>} [,])	(<i>n</i> ≥ 0)
<i>import</i>	→ <i>var</i>	
	<i>tycon</i> [(..) (<i>cname</i> ₁ , ... , <i>cname</i> _{<i>n</i>})]	(<i>n</i> ≥ 0)
	<i>tycls</i> [(..) (<i>var</i> ₁ , ... , <i>var</i> _{<i>n</i>})]	(<i>n</i> ≥ 0)
<i>cname</i>	→ <i>var</i> <i>con</i>	
<i>topdecls</i>	→ <i>topdecl</i> ₁ ; ... ; <i>topdecl</i> _{<i>n</i>}	(<i>n</i> ≥ 0)
<i>topdecl</i>	→ <i>type simpletype</i> = <i>type</i>	
	<i>data</i> [<i>context</i> =>] <i>simpletype</i> [= <i>constrs</i>] [<i>deriving</i>]	
	<i>newtype</i> [<i>context</i> =>] <i>simpletype</i> = <i>newconstr</i> [<i>deriving</i>]	
	<i>class</i> [<i>scontext</i> =>] <i>tycls</i> <i>tyvar</i> [<i>where cdecls</i>]	
	<i>instance</i> [<i>scontext</i> =>] <i>qtycls inst</i> [<i>where idecls</i>]	
	<i>default</i> (<i>type</i> ₁ , ... , <i>type</i> _{<i>n</i>})	(<i>n</i> ≥ 0)
	<i>foreign fdecl</i>	
	<i>decl</i>	
<i>decls</i>	→ { <i>decl</i> ₁ ; ... ; <i>decl</i> _{<i>n</i>} }	(<i>n</i> ≥ 0)
<i>decl</i>	→ <i>gdecl</i>	
	(<i>funlhs</i> <i>pat</i>) <i>rhs</i>	
<i>cdecls</i>	→ { <i>cdecl</i> ₁ ; ... ; <i>cdecl</i> _{<i>n</i>} }	(<i>n</i> ≥ 0)
<i>cdecl</i>	→ <i>gdecl</i>	
	(<i>funlhs</i> <i>var</i>) <i>rhs</i>	
<i>idecls</i>	→ { <i>idecl</i> ₁ ; ... ; <i>idecl</i> _{<i>n</i>} }	(<i>n</i> ≥ 0)
<i>idecl</i>	→ (<i>funlhs</i> <i>var</i>) <i>rhs</i>	
		(empty)
<i>gdecl</i>	→ <i>vars</i> :: [<i>context</i> =>] <i>type</i>	(type signature)
	<i>fixity</i> [<i>integer</i>] <i>ops</i>	(fixity declaration)
		(empty declaration)
<i>ops</i>	→ <i>op</i> ₁ , ... , <i>op</i> _{<i>n</i>}	(<i>n</i> ≥ 1)
<i>vars</i>	→ <i>var</i> ₁ , ... , <i>var</i> _{<i>n</i>}	(<i>n</i> ≥ 1)
<i>fixity</i>	→ <i>infixl</i> <i>infixr</i> <i>infix</i>	
<i>type</i>	→ <i>btype</i> [-> <i>type</i>]	(function type)

<i>btype</i>	→ <i>[btype] atype</i>	(type application)
<i>atype</i>	→ <i>gtycon</i> <i>tyvar</i> (<i>type</i> ₁ , ... , <i>type</i> _{<i>k</i>}) [<i>type</i>] (<i>type</i>)	(tuple type, <i>k</i> ≥ 2) (list type) (parenthesized constructor)
<i>gtycon</i>	→ <i>qtycon</i> () [] (->) (, { , })	(unit type) (list constructor) (function constructor) (tupling constructors)
<i>context</i>	→ <i>class</i> (<i>class</i> ₁ , ... , <i>class</i> _{<i>n</i>})	(<i>n</i> ≥ 0)
<i>class</i>	→ <i>qtycls tyvar</i> <i>qtycls</i> (<i>tyvar atype</i> ₁ ... <i>atype</i> _{<i>n</i>})	(<i>n</i> ≥ 1)
<i>scontext</i>	→ <i>simpleclass</i> (<i>simpleclass</i> ₁ , ... , <i>simpleclass</i> _{<i>n</i>})	(<i>n</i> ≥ 0)
<i>simpleclass</i>	→ <i>qtycls tyvar</i>	
<i>simpletype</i>	→ <i>tycon tyvar</i> ₁ ... <i>tyvar</i> _{<i>k</i>}	(<i>k</i> ≥ 0)
<i>constrs</i>	→ <i>constr</i> ₁ ... <i>constr</i> _{<i>n</i>}	(<i>n</i> ≥ 1)
<i>constr</i>	→ <i>con</i> [!] <i>atype</i> ₁ ... [!] <i>atype</i> _{<i>k</i>} (<i>btype</i> ! <i>atype</i>) <i>conop</i> (<i>btype</i> ! <i>atype</i>) <i>con</i> { <i>fielddecl</i> ₁ , ... , <i>fielddecl</i> _{<i>n</i>} }	(arity <i>con</i> = <i>k</i> , <i>k</i> ≥ 0) (infix <i>conop</i>) (<i>n</i> ≥ 0)
<i>newconstr</i>	→ <i>con atype</i> <i>con</i> { <i>var</i> :: <i>type</i> }	
<i>fielddecl</i>	→ <i>vars</i> :: (<i>type</i> ! <i>atype</i>)	
<i>deriving</i>	→ <i>deriving</i> (<i>dclass</i> (<i>dclass</i> ₁ , ... , <i>dclass</i> _{<i>n</i>}))	(<i>n</i> ≥ 0)
<i>dclass</i>	→ <i>qtycls</i>	
<i>inst</i>	→ <i>gtycon</i> (<i>gtycon tyvar</i> ₁ ... <i>tyvar</i> _{<i>k</i>}) (<i>tyvar</i> ₁ , ... , <i>tyvar</i> _{<i>k</i>}) [<i>tyvar</i>] (<i>tyvar</i> ₁ -> <i>tyvar</i> ₂)	(<i>k</i> ≥ 0, <i>tyvars</i> distinct) (<i>k</i> ≥ 2, <i>tyvars</i> distinct) <i>tyvar</i> ₁ and <i>tyvar</i> ₂ distinct
<i>fdecl</i>	→ <i>import callconv</i> [<i>safety</i>] <i>impent var</i> :: <i>ftype</i> <i>export callconv expent var</i> :: <i>ftype</i>	(define variable) (expose variable)
<i>callconv</i>	→ <i>ccall</i> <i>stdcall</i> <i>cplusplus</i> <i>jvm</i> <i>dotnet</i> system-specific calling conventions	(calling convention)
<i>impent</i>	→ [<i>string</i>]	(see Section 8.5.1)
<i>expent</i>	→ [<i>string</i>]	(see Section 8.5.1)
<i>safety</i>	→ <i>unsafe</i> <i>safe</i>	
<i>ftype</i>	→ <i>frtype</i> <i>fatype</i> → <i>ftype</i>	
<i>frtype</i>	→ <i>fatype</i> ()	
<i>fatype</i>	→ <i>qtycon atype</i> ₁ ... <i>atype</i> _{<i>k</i>}	(<i>k</i> ≥ 0)
<i>funlhs</i>	→ <i>var apat</i> { <i>apat</i> } <i>pat varop pat</i>	

	(<i>funlhs</i>) <i>apat</i> { <i>apat</i> }	
<i>rhs</i>	→ = <i>exp</i> [where <i>decls</i>] <i>gdrhs</i> [where <i>decls</i>]	
<i>gdrhs</i>	→ <i>guards</i> = <i>exp</i> [<i>gdrhs</i>]	
<i>guards</i>	→ <i>guard</i> ₁ , ..., <i>guard</i> _{<i>n</i>}	(<i>n</i> ≥ 1)
<i>guard</i>	→ <i>pat</i> <- <i>infixexp</i> <i>let decls</i> <i>infixexp</i>	(pattern guard) (local declaration) (boolean guard)
<i>exp</i>	→ <i>infixexp</i> :: [context =>] <i>type</i> <i>infixexp</i>	(expression type signature)
<i>infixexp</i>	→ <i>lexp</i> <i>qop</i> <i>infixexp</i> - <i>infixexp</i> <i>lexp</i>	(infix operator application) (prefix negation)
<i>lexp</i>	→ \ <i>apat</i> ₁ ... <i>apat</i> _{<i>n</i>} -> <i>exp</i> <i>let decls</i> in <i>exp</i> if <i>exp</i> [;] then <i>exp</i> [;] else <i>exp</i> case <i>exp</i> of { <i>alts</i> } do { <i>stmts</i> } <i>fexp</i>	(lambda abstraction, <i>n</i> ≥ 1) (let expression) (conditional) (case expression) (do expression)
<i>fexp</i>	→ [<i>fexp</i>] <i>aexp</i>	(function application)
<i>aexp</i>	→ <i>qvar</i> <i>gcon</i> <i>literal</i> (<i>exp</i>) (<i>exp</i> ₁ , ... , <i>exp</i> _{<i>k</i>}) [<i>exp</i> ₁ , ... , <i>exp</i> _{<i>k</i>}] [<i>exp</i> ₁ [, <i>exp</i> ₂] .. [<i>exp</i> ₃]] [<i>exp</i> <i>qual</i> ₁ , ... , <i>qual</i> _{<i>n</i>}] (<i>infixexp</i> <i>qop</i>) (<i>qop</i> _{-} <i>infixexp</i>) <i>qcon</i> { <i>fbind</i> ₁ , ... , <i>fbind</i> _{<i>n</i>} } <i>aexp</i> _{<i>qcon</i>} { <i>fbind</i> ₁ , ... , <i>fbind</i> _{<i>n</i>} }	(variable) (general constructor) (parenthesized expression) (tuple, <i>k</i> ≥ 2) (list, <i>k</i> ≥ 1) (arithmetic sequence) (list comprehension, <i>n</i> ≥ 1) (left section) (right section) (labeled construction, <i>n</i> ≥ 0) (labeled update, <i>n</i> ≥ 1)
<i>qual</i>	→ <i>pat</i> <- <i>exp</i> <i>let decls</i> <i>exp</i>	(generator) (local declaration) (guard)
<i>alts</i>	→ <i>alt</i> ₁ ; ... ; <i>alt</i> _{<i>n</i>}	(<i>n</i> ≥ 1)
<i>alt</i>	→ <i>pat</i> -> <i>exp</i> [where <i>decls</i>] <i>pat</i> <i>gdpat</i> [where <i>decls</i>] 	 (empty alternative)
<i>gdpat</i>	→ <i>guards</i> -> <i>exp</i> [<i>gdpat</i>]	
<i>stmts</i>	→ <i>stmt</i> ₁ ... <i>stmt</i> _{<i>n</i>} <i>exp</i> [;]	(<i>n</i> ≥ 0)
<i>stmt</i>	→ <i>exp</i> ; <i>pat</i> <- <i>exp</i> ; <i>let decls</i> ; ;	 (empty statement)

<i>fbind</i>	→ <i>qvar</i> = <i>exp</i>	
<i>pat</i>	→ <i>lpat</i> <i>qconop</i> <i>pat</i> <i>lpat</i>	(infix constructor)
<i>lpat</i>	→ <i>apat</i> - (<i>integer</i> <i>float</i>) <i>gcon</i> <i>apat</i> ₁ ... <i>apat</i> _{<i>k</i>}	(negative literal) (arity <i>gcon</i> = <i>k</i> , <i>k</i> ≥ 1)
<i>apat</i>	→ <i>var</i> [@ <i>apat</i>] <i>gcon</i> <i>qcon</i> { <i>fpat</i> ₁ , ... , <i>fpat</i> _{<i>k</i>} } <i>literal</i> - (<i>pat</i>) (<i>pat</i> ₁ , ... , <i>pat</i> _{<i>k</i>}) [<i>pat</i> ₁ , ... , <i>pat</i> _{<i>k</i>}] ~ <i>apat</i>	(as pattern) (arity <i>gcon</i> = 0) (labeled pattern, <i>k</i> ≥ 0) (wildcard) (parenthesized pattern) (tuple pattern, <i>k</i> ≥ 2) (list pattern, <i>k</i> ≥ 1) (irrefutable pattern)
<i>fpat</i>	→ <i>qvar</i> = <i>pat</i>	
<i>gcon</i>	→ () [] (, { , }) <i>qcon</i>	
<i>var</i>	→ <i>varid</i> (<i>varsym</i>)	(variable)
<i>qvar</i>	→ <i>qvarid</i> (<i>qvarsym</i>)	(qualified variable)
<i>con</i>	→ <i>conid</i> (<i>consym</i>)	(constructor)
<i>qcon</i>	→ <i>qconid</i> (<i>gconsym</i>)	(qualified constructor)
<i>varop</i>	→ <i>varsym</i> ` <i>varid</i> `	(variable operator)
<i>qvarop</i>	→ <i>qvarsym</i> ` <i>qvarid</i> `	(qualified variable operator)
<i>conop</i>	→ <i>consym</i> ` <i>conid</i> `	(constructor operator)
<i>qconop</i>	→ <i>gconsym</i> ` <i>qconid</i> `	(qualified constructor operator)
<i>op</i>	→ <i>varop</i> <i>conop</i>	(operator)
<i>qop</i>	→ <i>qvarop</i> <i>qconop</i>	(qualified operator)
<i>gconsym</i>	→ : <i>qconsym</i>	

10.6 Fixity Resolution

The following is an example implementation of fixity resolution for Haskell expressions. Fixity resolution also applies to Haskell patterns, but patterns are a subset of expressions so in what follows we consider only expressions for simplicity.

The function `resolve` takes a list in which the elements are expressions or operators, i.e. an instance of the *infixexp* non-terminal in the context-free grammar. It returns either `Just e` where *e* is the resolved expression, or `Nothing` if the input does not represent a valid expression. In a compiler, of course, it would be better to return more information about the operators involved for the purposes of producing a useful error message, but the `Maybe` type will suffice to illustrate the algorithm here.

```
import Control.Monad

type Prec    = Int
type Var     = String

data Op = Op String Prec Fixity
  deriving (Eq, Show)

data Fixity = Leftfix | Rightfix | Nonfix
  deriving (Eq, Show)
```

```

data Exp = Var Var | OpApp Exp Op Exp | Neg Exp
  deriving (Eq,Show)

data Tok = TExp Exp | TOp Op | TNeg
  deriving (Eq,Show)

resolve :: [Tok] -> Maybe Exp
resolve tokens = fmap fst $ parseNeg (Op "" (-1) Nonfix) tokens
  where
    parseNeg :: Op -> [Tok] -> Maybe (Exp,[Tok])
    parseNeg op1 (TExp e1 : rest)
      = parse op1 e1 rest
    parseNeg op1 (TNeg : rest)
      = do guard (prec1 < 6)
          (r, rest') <- parseNeg (Op "-" 6 Leftfix) rest
          parse op1 (Neg r) rest'
    where
      Op _ prec1 fix1 = op1

    parse :: Op -> Exp -> [Tok] -> Maybe (Exp, [Tok])
    parse _ e1 [] = Just (e1, [])
    parse op1 e1 (TOp op2 : rest)
      -- case (1): check for illegal expressions
      | prec1 == prec2 && (fix1 /= fix2 || fix1 == Nonfix)
      = Nothing

      -- case (2): op1 and op2 should associate to the left
      | prec1 > prec2 || (prec1 == prec2 && fix1 == Leftfix)
      = Just (e1, TOp op2 : rest)

      -- case (3): op1 and op2 should associate to the right
      | otherwise
      = do (r,rest') <- parseNeg op2 rest
          parse op1 (OpApp e1 op2 r) rest'
    where
      Op _ prec1 fix1 = op1
      Op _ prec2 fix2 = op2

```

The algorithm works as follows. At each stage we have a call

```
parse op1 E1 (op2 : tokens)
```

which means that we are looking at an expression like

$$E0 \text{ 'op1' } E1 \text{ 'op2' } \dots \quad (1)$$

(the caller holds $E0$). The job of `parse` is to build the expression to the right of `op1`, returning the expression and any remaining input.

There are three cases to consider:

1. if `op1` and `op2` have the same precedence, but they do not have the same associativity, or they are declared to be nonfix, then the expression is illegal.
2. If `op1` has a higher precedence than `op2`, or `op1` and `op2` should left-associate, then we know that the expression to the right of `op1` is $E1$, so we return this to the caller.
3. Otherwise, we know we want to build an expression of the form $E1 \text{ 'op2' } R$. To find R , we call `parseNeg op2 tokens` to compute the expression to the right of `op2`, namely R (more about `parseNeg` below, but essentially if `tokens` is of the form $(E2 : \text{rest})$, then this is equivalent to `parse op2 E2 rest`). Now, we have $E0 \text{ 'op1' } (E1 \text{ 'op2' } R) \text{ 'op3' } \dots$ where `op3` is the next operator in the input. This is an instance of (1) above, so to continue we call `parse`, with the new $E1 == (E1 \text{ 'op2' } R)$.

To initialise the algorithm, we set `op1` to be an imaginary operator with precedence lower than anything else. Hence `parse` will consume the whole input, and return the resulting expression.

The handling of the prefix negation operator, `-`, complicates matters only slightly. Recall that prefix negation has the same fixity as infix negation: left-associative with precedence 6. The operator to the left of `-`, if there is

one, must have precedence lower than 6 for the expression to be legal. The negation operator itself may left-associate with operators of the same fixity (e.g. $+$). So for example $-a + b$ is legal and resolves as $(-a) + b$, but $a + -b$ is illegal.

The function `parseNeg` handles prefix negation. If we encounter a negation operator, and it is legal in this position (the operator to the left has precedence lower than 6), then we proceed in a similar way to case (3) above: compute the argument to $-$ by recursively calling `parseNeg`, and then continue by calling `parse`.

Note that this algorithm is insensitive to the range and resolution of precedences. There is no reason in principle that Haskell should be limited to integral precedences in the range 1 to 10; a larger range, or fractional values, would present no additional difficulties.